

Lab 4 - VibeCoding: LLM chat history

Name: B V TEJAS SRN: PES1UG24CS108 Section: B

Chat link: <https://claude.ai/share/de226c97-b7b7-4bd0-80b1-868cdf862069>

Prompt 1 (Task 1)

have a Pygame Fruit Ninja game (files below). In game/fruit.py, contains_point(x, y) only tests whether a single mouse position lies inside the fruit's circle, and _handle_motion in game/game_engine.py calls it once per MOUSEMOTION event. A fast swipe jumps from outside the fruit to past it between two consecutive events, so the fruit is never sliced even though the blade visually crossed it.

Fix it by testing the line segment between the previous and current mouse positions against the fruit's circle. Add an intersects_segment(x1, y1, x2, y2) method to Fruit using point-to-segment distance, and have _handle_motion track the last position and use it. Return the complete updated fruit.py and game_engine.py.

[Attached files]

Claude - response to prompt 1

I'll pull the current fruit.py and game_engine.py out of the attached handout first, then make the swept-segment fix.

[Ran 6 commands, edited 2 files, read 2 files]

Fast swipes now slice: both updated files are attached as fruit.py and game_engine.py, drop-in replacements for the ones in game/.

The code wasn't in your message (only the two PDFs came through), so I worked from the starter in SETAPESU26/41_fruit-ninja from the repo list. If your copy has already diverged from that starter, send me your versions and I'll reapply the change.

fruit.py

- Added intersects_segment(x1, y1, x2, y2), which finds the point on the segment closest to the fruit's centre and checks whether it is within the radius.
- A swipe that stops short of the fruit does not count, and a zero-length segment falls back to contains_point.

game_engine.py

- __init__ gets self._last_pos = None.
- _handle_motion tests the segment from _last_pos to the current position, then stores the current position. On the first event there is no previous point, so it tests just the current position.
- One addition you didn't ask for: handle_event resets _last_pos and clears the trail when the pointer leaves the window. Without it, leaving on one side and re-entering on another would join the two points into one long blade and slice everything between them, bombs included.

I ran it headless against the original: a swipe from (200, 300) to (500, 300) across a fruit at (350, 300) is missed by the starter and sliced by the fix. Grazing, tangent, near-miss, stop-short, bomb and window-leave cases also pass. I couldn't play it with a real mouse here, so do a quick live check when you record your "after" video.

For Task 2: the engine keeps processing mouse motion after game over, so the score can still climb on the frozen fruit. You'll want to gate slicing on not self.game_over when you build the game-over screen.

Prompt 2 (Task 2)

Task 2: add a proper game-over screen. Work from the fruit.py and game_engine.py you just gave me.

Right now, when the game ends, render() only prints "Game over!" to the console. Replace that with a real on-screen game-over screen:

- Dim the playfield with a semi-transparent dark overlay.
- Show "GAME OVER" in large text, centred.
- Show "Final Score: N" below it, centred.
- Remove the console print and the _game_over_logged flag.
- Once game_over is True, stop slicing: _handle_motion must not slice fruit or change the score or lives. It's fine to keep drawing the trail.

Keep the swept-segment slice detection working. Don't add replay or sound yet. Return the complete updated game_engine.py, and tell me if any other file needs to change.

Claude - response to prompt 2

I'll add the overlay and the slicing gate to game_engine.py, then render a game-over frame headless to check it.

[Edited a file, read 2 files, ran 2 commands]

The game-over screen is in the attached `game_engine.py`; no other file needs to change - `fruit.py` and `main.py` stay exactly as they are. `game_over_preview.png` shows a frame of it rendered from a headless test scene.

What changed in `game_engine.py`:

- Overlay: a semi-transparent black surface is built once in `__init__` and drawn last in `render()`, so it dims the fruit, trail and score/lives display.
- Text: "GAME OVER" in 72pt bold and "Final Score: N" in 36pt below it, both centred on the window. This lives in a new `_render_game_over()` method.
- Console output: the print and the `_game_over_logged` flag are gone.
- Slicing gate: `_handle_motion` skips the slice loop once `game_over` is True, so score and lives are frozen. It still records the position, so the trail keeps drawing.
- One addition you didn't ask for: the slice loop now stops as soon as a bomb is hit, so a fruit crossed by the same swipe after the bomb isn't scored once the game has ended.

Swept-segment detection is untouched while playing, and the Task 1 tests still pass. I also checked headless that both endings (bomb, and lives running out) show the screen, that swiping across fruit afterwards changes nothing, and that nothing is printed. As before, I couldn't play it with a real mouse, so give it a quick live run. The screen has no key handling yet, so the window just stays on the game-over screen until you close it. That is where Task 3's replay prompt will go.

Prompt 3 (Task 3)

Task 3: add replay with difficulty selection, or exit. Work from the `game_engine.py` you just gave me.

On the game-over screen:

- Show the options below the final score: "E - Easy M - Medium H - Hard Q - Quit".
- Pressing E, M or H starts a new game on that difficulty. Pressing Q closes the game cleanly.
- Each difficulty sets `spawn_interval`, `bomb_chance` and `speed_scale`. Medium must match the current values (55, 0.15, 1.0); Easy is slower with fewer bombs, Hard is faster with more bombs.
- Restart must fully reset score, lives, fruits, trail, the spawn timer, `_last_pos` and `game_over`.
- The keys must do nothing while a game is in progress.
- The first game still starts straight away on Medium, as it does now.

Keep the swept-segment slicing and the game-over overlay working. Don't add sound yet. Return the complete updated `game_engine.py`, and `main.py` too if it needs to change for quitting.

Claude - response to prompt 3

I'll add the difficulty presets, restart and quit handling to `game_engine.py`, then test it headless, including a run through your real `main.py` loop to confirm Q shuts down cleanly.

[Edited a file, read 2 files, ran 2 commands]

Replay and quit are in the attached `game_engine.py`, and `main.py` does not need to change. Pressing Q posts pygame's own QUIT event, so your existing loop handles it exactly like the window's close button and then calls `pygame.quit()`.

What changed in `game_engine.py`:

- Menu line: "E - Easy M - Medium H - Hard Q - Quit" sits centred under the final score; `game_over_menu_preview.png` shows the layout. I moved the title and score up 20px to make room.
- Restart: a new `start_game(difficulty)` resets score, lives, fruits, trail, the spawn timer, `_last_pos` and `game_over`. `__init__` calls `start_game("medium")`, so the first game and every replay go through the same code.
- Keys: `_handle_key` returns immediately unless `game_over` is True, so E, M, H and Q do nothing during play.

The difficulty values, which I picked (only Medium was specified):

Difficulty | `spawn_interval` | `bomb_chance` | `speed_scale`

Easy | 80 | 0.08 | 0.9

Medium | 55 | 0.15 | 1.0

Hard | 35 | 0.25 | 1.2

One thing to know about Easy: `speed_scale` only multiplies launch speed, so slower fruit also peak lower and are in the air about 10% less time than on Medium. Easy is still easier overall on spawn rate and bombs, but if it feels cramped when you play, set its `speed_scale` back to 1.0 in the DIFFICULTIES table at the top of the file.

I tested it headless, including through your real main.py loop: Q mid-game is ignored, H restarts on Hard with everything reset, and Q on the game-over screen exits the loop and shuts pygame down. The Task 1 and Task 2 tests still pass. I haven't played it with a real keyboard and mouse, so give it a live run before recording.

Prompt 4 (Task 4)

Task 4: add sound effects. Work from the game_engine.py you just gave me.

- Play a short sound when a fruit is sliced, a different one when a bomb is sliced, and a game-over sound when the game ends (either by bomb or by running out of lives). The game-over sound must play once per game, not every frame.
- Generate the sounds in code (for example short tones or noise bursts built with Python's array module or pygame.sndarray), so no audio files have to be added to the repo.
- If the mixer can't be initialised (no audio device), the game must still run normally with sound disabled, not crash.
- Don't play any sound for swipes after game over.

Keep slicing, the game-over screen and replay/quit working exactly as they are. Return the complete updated game_engine.py, and requirements.txt and main.py too if either needs to change.

Claude - response to prompt 4

I'll add generated sound effects to game_engine.py (built with the array module, so no numpy), then test with a dummy audio device and with no audio device at all.

[Edited a file, read a file, ran a command]

Sound effects are in the attached game_engine.py; requirements.txt and main.py don't need to change. I couldn't listen to the sounds here, so I checked their lengths and pitches numerically - play it once and judge them by ear.

- No new dependency: the sounds are built with Python's standard array module rather than pygame.sndarray, which would have needed numpy.
- Fruit sliced: a 0.09 s rising "swish" (700 to 1700 Hz).
- Bomb sliced: a 0.45 s low boom (muffled noise plus a 55 Hz thump).
- Game over: three falling notes (G, E, C), 0.76 s in total.
- Once per game: a new _end_game() is the only place game_over is switched on, and it plays the game-over sound there. Both the bomb path and the lives-out path go through it.
- No audio device: if the mixer can't start, _build_sounds() returns nothing and _play() silently does nothing. self.sound_enabled tells you which case you're in.
- After game over: no sound, because slicing is already gated there.

Slicing a bomb starts the boom and the game-over notes at the same moment, so they overlap. If that sounds muddy, tell me and I'll make the notes wait until the boom finishes.

Tested headless:

- With a dummy audio device: each event triggers the right sound the right number of times, including across restarts.
- With no audio device (three different ways): the game runs through slicing, both endings, restart and quit without crashing, including through your real main.py loop.
- Earlier tasks: the Task 1-3 tests still pass.

If the pitch or length of any sound is off for you, the three _..._samples functions at the top of the file are where to change them.